Centro Universitário de Excelência
Sistemas de Informação

Construindo a Web API Fundacional da MecâniQA

Autores: Gustavo Prates Caetano
Nallanda Novato
Matheus Costa Rodrigues
Thainá Salgado
Vitor Hugo

Vitória da Conquista, 2026

Agenda

O objetivo desta apresentação é demonstrar a evolução da MecâniQA na OAT 3: a migração para um banco de dados real (MySQL) via JDBC puro, tratamento de erros e o isolamento das regras na nova Camada de Serviço.

MySQL
Migração para banco real e versionamento de scripts.

2. JDBC Nativo
Construção manual de queries. Zero ferramentas ORM.

2. JDBC Nativo
Construção manual de queries. Zero ferramentas ORM.

5.Camada Service
Isolamento obrigatório entre regras e controladores.

3. ResultSets
Criação de mappers para conversão segura de dados.

O que muda para a MecâniQA?

O impacto direto desta entrega técnica no dia a dia da oficina.

Fim da Perda de Dados (Memória Volátil)

Ordens de Serviço e controle de Peças ficam salvos permanentemente e imunes a reinicializações.

Regras de Negócio Blindadas

A nova camada Service centraliza validações e protege a oficina contra dados inválidos.

Infraestrutura Confiável

A modernização ganha escala ao operar sobre um banco de dados real e amplamente adotado no mercado.

Motor de Armazenamento

Adoção do MySQL
Banco de dados relacional oficial do sistema.

Proibição Estrita de ORM
Zero ferramentas automatizadas (JPA ou Hibernate).

Versionamento de Scripts
Controle rigoroso das instruções DDL e DML no repositório.

```
CREATE TABLE IF NOT EXISTS peca (
    id                      BIGINT        NOT NULL AUTO_INCREMENT,
    nome                    VARCHAR(150)  NOT NULL,
    codigo_barras           VARCHAR(50),
    fornecedor_marca        VARCHAR(100),
    quantidade_em_estoque   INT           NOT NULL DEFAULT 0,
    preco_custo             DECIMAL(10,2),
    preco_venda             DECIMAL(10,2),
    data_cadastro           DATETIME,
    data_ultima_atualizacao DATETIME,
    categoria               VARCHAR(20),
    tamanho                 VARCHAR(30),
    cor                     VARCHAR(30),
    PRIMARY KEY (id)
    ) ENGINE=InnoDB;
```

Repositórios e JDBC

Itens implementados:

Molde Único: Interface genérica para padronizar o acesso a qualquer tabela.

Operações Obrigatórias: Contratos fixos para salvar, buscar e deletar dados.

SQL Manual (JDBC): Comandos nativos direto no Java sem ferramentas "mágicas".

```java
@Override
protected void aposInserir(Connection conexao, OrdemServico os) throws SQLException {
    String sqlServico = "INSERT INTO ordem_servico_servico (ordem_servico_id, servico_id) VALUES (?, ?)";
    for (Servico servico : os.getServicos()) {
        try (PreparedStatement ps = conexao.prepareStatement(sqlServico)) {
            ps.setLong(1, os.getCodigoUnico());
            ps.setLong(2, servico.getCodigoUnico());
            ps.executeUpdate();
        }
    }

    String sqlPeca = "INSERT INTO ordem_servico_peca (ordem_servico_id, peca_id, quantidade) VALUES (?, ?, ?)";
    for (ItemPedidoPeca item : os.getPecasUtilizadas()) {
        try (PreparedStatement ps = conexao.prepareStatement(sqlPeca)) {
            ps.setLong(1, os.getCodigoUnico());
            ps.setLong(2, item.getPeca().getCodigoUnico());
            ps.setInt(3, item.getQuantidade());
            ps.executeUpdate();
```

Mappers e Exceções

Conversão Prática: Transformamos os dados brutos do banco em objetos do sistema.

Erros Sob Medida: Criamos classes próprias de erro para proteger as regras da oficina.

Controle Manual: Tratamos as falhas com try/catch, sem usar atalhos globais do Spring.

```
private OrdemServicoRowMapper() {
}

public static OrdemServico mapear(ResultSet rs) throws SQLException {
    return OrdemServico.builder()
            .comCodigo(rs.getLong("id"))
            .comPlaca(rs.getString("placa_veiculo"))
            .comDescricao(rs.getString("descricao_problema"))
            .comStatus(JdbcUtil.lerEnum(rs, "status", StatusOS.class))
            .comDataAbertura(JdbcUtil.lerDataHora(rs, "data_abertura"))
            .build();
}
```

```
public class OrdemServicoNaoEncontradaException extends RecursoNaoEncontradoException {

    public OrdemServicoNaoEncontradaException(Long id) {
        super("Ordem de Serviço não encontrada para o código: " + id);
    }
}
```

A Nova Camada de Serviço

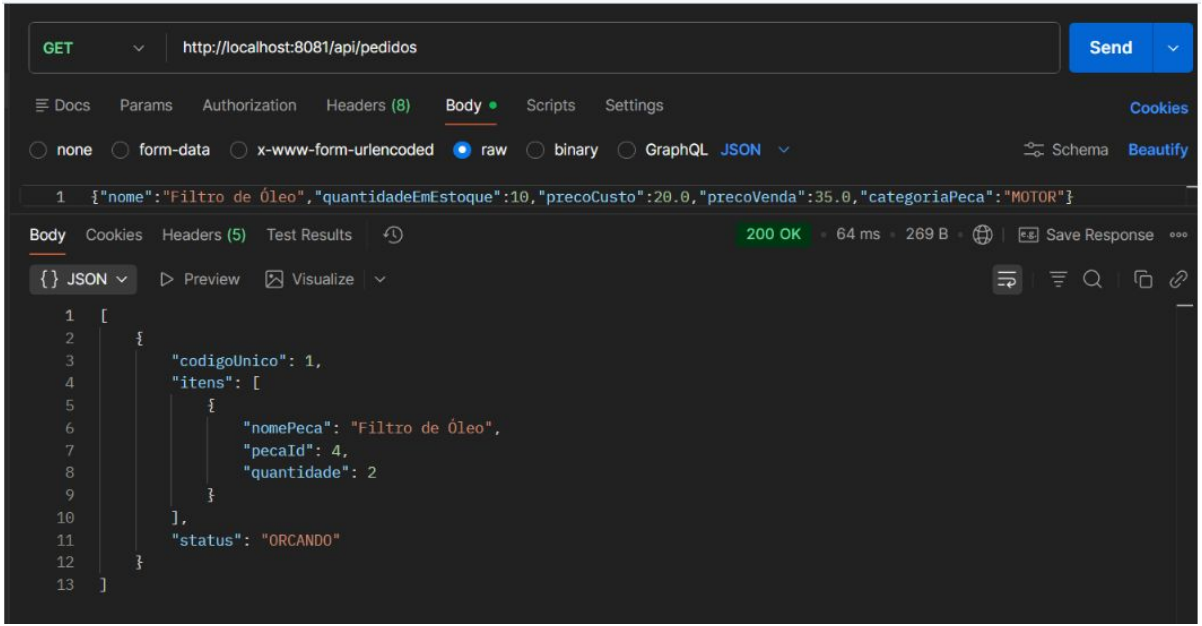


A Regra de Ouro da Arquitetura

Isolamento total. Controladores repassam DTOs. Regras de negócio, validações e comunicação JDBC ocorrem **exclusivamente** na camada de Service.

Testes na Prática (Postman)

Validando o sucesso das requisições passando por Controller, Service, e persistindo no banco MySQL.



OAT 3 - Validação de Ponta a Ponta

Considerações finais

- **Persistência Sólida:** Entendimento profundo de comunicação com o banco sem abstrações mágicas.
- **Tratamento Robusto:** Captura de exceções manuais garantindo integridade nos retornos HTTP.
- **Maturidade Arquitetural:** Divisão clara de responsabilidades preparando o terreno para manutenções futuras.

Referências

- Documentação Oficial do MySQL 8.0 Reference Manual.
- Documentação Oficial Java SE - API JDBC (Java Database Connectivity).
- Centro Universitário de Excelência (Unex). *OAT 3 - Persistindo os meus Dados*. Vitória da Conquista, 2026.